

National University of Singapore
School of Computing

Semester 1, AY2022-23

CS4246/CS5446

AI Planning and Decision Making

Issued: 14 Oct 2022

Due: 1 Nov 2022@23:59

Assignment 3

General Guidelines

Please complete the Assignment in the **same 2-person TEAM** as the past assignment(s). **ONE person from each team should submit ALL the answers (for both programming and homework problems)**. Grading will be done team-wise.

On collaboration

- You are encouraged to discuss solution ideas. However, each team *must write up the solutions independently*. It is considered as plagiarism if the solution write-up is highly similar to other teams's write-ups or to other sources.

Homework Problems Submission

- Submission will take place through **LumiNUS Quiz**.
- **No late submissions for the homework assignment solutions are allowed.**

Programming Problems Submission

- **ONE person from each team should submit the solutions.** Grading will be done team-wise.
- **IMPORTANT!** Please write the following on the top of your solution files for EACH problem in the assignment:
 - Name, metric number and Tutorial Group of **all team members** as they appear on LumiNUS
 - Collaborators (write **None** if no collaborators)
 - Sources, if any, or if you obtained the solutions through research, e.g. through the web.
- **Upload the code of your programming assignment to the aiVLE evaluation server**, by following the instructions [here](#). Please keep a copy of your submission zip file for safety.
- Please name your files, using ONE of the members' information according to this format:
A3_<Member1-TutorialGroup>-<Member1-name>-Task
E.g., A3-TG6-xiaomei.tan-Task.zip
- **A late score penalty of 20% will be incurred for late programming assignment solutions submissions.**

1 Homework Assignment (LumiNUS Quiz)

1.1 Reinforcement Learning

1. [4 marks] Write out the parameter update equations for TD learning with the utility of state represented using the following approximator.

$$\hat{U}(x, y) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 \sqrt{x^2 - y^2 + 3xy}$$

2. [6 marks] Assume each $\theta_i = 1$ initially. The agent starts in state $(1, 1)$. There is only one action MOVE and each transition yields a reward of $\sqrt{3}$. **Round off your answers to two decimal places.**
 - a) On executing the action MOVE, the agent transits to state $(2, 1)$. What are the updated values of θ_i ?
 - b) After the above step, what are the updated values of states $(1, 1)$ and $(2, 1)$?
3. [5 marks] Consider the grid-world problem that we have seen in the lectures. We have used the state's (x, y) coordinates in the factored state representation. However, maintaining the states is not feasible in large problems. You are tasked to extend the grid world problem to unbounded dimensions (e.g., see Figure 1. This means, you don't know the X_{max}, Y_{max} values. You have multiple terminal states with $+1$ or -1 rewards. There can be many obstacles (black squares) in the environment.
What are the features you will use to model this generalized learning problem? (List 5 or more features)

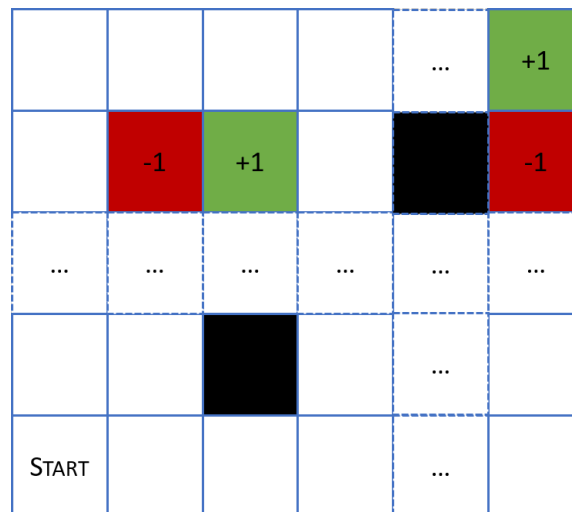


Figure 1: Extended grid world

1.2 POMDP

1. **[4 marks]** Consider a problem with two states s_1 and s_2 , current belief $b(s_1) = 0.6$ and $b(s_2) = 0.4$. For action a . The transition probabilities are as follows:

- $P(s_1|s_1, a) = 0.3$; $P(s_2|s_1, a) = 0.7$
- $P(s_1|s_2, a) = 0.6$; $P(s_2|s_2, a) = 0.4$

The observation probabilities are as shown below:

- The observation o_1 reports that the agent is in state s_1 and agent is in state s_1 with 80% probability.
- The observation o_1 reports that the agent is in state s_1 but agent is in state s_2 with 20% probability.

What is the updated belief after receiving the evidence $e = o_1$. **Round off your answers to three decimal places.**

2. **[6 marks] Human tracking with onboard camera.** An onboard camera on a mobile robot can be controlled to view different parts of the environment through the pan and tilt actions. If there is a human in the environment, the camera should point to the person and track the person's movement. For simplicity, assume there is always one person in the environment and the robot is stationary.

The camera uses positive values to pan right or tilt up and -ve values to pan left or tilt down. Each pan command pan's the camera by 45° . Each tilt command tilt's the camera by 15° . When the camera pans to the right, there is a probability of 0.2 that it reaches the next subsequent "view". Tilt is noisy, there is only 40% chance that the correct tilt angle is obtained.

- a) Specify the problem as a POMDP. In particular, specify the variables that define the state space, the actions, the transition functions using these state variables defined.
 - b) What is the observation space and the observation function?
-

2 Programming Assignment (aiVLE submission)

In Assignment 2, we have learned to solve non-deterministic planning problems with Value Iteration and Monte Carlo Tree Search. However, both methods have a problem with scaling on a larger task. In this task, we will learn to solve slightly larger planning problems in deterministic environments with *Deep Q Learning*. Despite the fact that we are solving a deterministic problem, the method that we are about to learn can also solve non-deterministic problems. We chose to limit the task to be deterministic just for simplicity and better predictability during training.

We will be using the same `gym_grid_environment` (<https://github.com/cs4246/gym-grid-driving>) to simulate the solutions. All dependencies have been installed in the docker image which you have downloaded.

By now, you should be familiar with the `gym_grid_environment`. Go through the IPython Notebook file on Google Colab [here](#) if you have not already done so in the previous programming assignment. You are now ready to begin solving the task.

Programming problem: Deep Q Learning

[25 marks]

Problem Definition

In the previous two homeworks, we assume that we somehow have the model of the environment. PDDL requires the determinized version of the MDP, VI requires the MDP, and MCTS requires a simulator of the environment. Here, we want to tackle a problem in which we don't have the underlying details of the environment. What we have instead is just the observation (image) of the top view of the street. Fortunately, DQN can be trained without any of those details.

In this task, we will implement a DQN algorithm with the following features.

- *Replay buffer*: During the training, the DQN agent will interact with the environment and produce a sequence of transition (experiences) that can be used for training data. However, training directly with such experiences may lead to instability due to the data being temporally correlated. To remove correlation in the data, experiences are instead stored in a memory buffer. At every training step, the experiences are then sampled to train the agent. The memory is often called replay buffer since it contains the “replay” of the agent's interaction.
- *Epsilon greedy*: Training a DQN agent (and RL agent in general) is highly dependent on the ability to find good regions (i.e., regions at which it yields high value). If good regions fail to be discovered during training, the agent will be unlikely to perform well. However, most of the time, we won't know of a good heuristic to determine which region is useful for training, hence random exploration strategy is employed. One such strategy is called the epsilon greedy exploration strategy, in which the agent chooses to perform a random action with some probability at every step during experience gathering.

- *Target Network*: DQN loss function computes the distance between the Q values induced by the model with its expected Q values (given by the next state Q values and current state reward). However, such an objective is hard to optimize because it optimizes for a moving target distribution (i.e., the expected Q values). To minimize such a problem, instead of computing the expected Q values using the model that we are currently training, we use a target model (target network) instead, a replica of the model that is synchronized every few episodes.

We provide an agent template that has a code snippet that allows you to train the DQN agent and test the agent in the server. The use of the agent template is required. Your agent definition is located in the `agent/dqn.py`. What you have to do is to fill all functions that are marked with “FILL ME” listed below.

- **Replay Buffer functions**:
 - `ReplayBuffer.__init__`: initialize the replay buffer.
 - `ReplayBuffer.push`: add transition to replay buffer.
 - `ReplayBuffer.sample`: sample from the replay buffer.
- `BaseAgent.act`: computes an action of a given state following the ϵ -greedy formulation: with $(1 - \epsilon)$ probability output the actual action, otherwise, with ϵ probability output random action.
- `compute_loss`: compute the temporal difference loss δ of the DQN model Q , incorporating the target network $\hat{Q}$: $\delta = Q(s, a) - (r + \gamma \max_a \hat{Q}(s', a))$; with discounting factor γ and reward r of state s after taking action a and giving next state s' . To minimize this distance, we can use [Mean Squared Error \(MSE\) loss](#) or [Huber loss](#). Huber loss is known to be less sensitive to outliers and in some cases prevents exploding gradients (e.g. see Fast R-CNN paper by Ross Girshick).

You can see more details in the `agent/dqn.py` code comments.

Training & Testing the Agent

You can train the agent by executing `python dqn.py --train`. It will train the agent neural network model for 2,000 episodes, with 10 training steps for each episode. The training will take about 6-10 minutes in Tembusu (See [2](#)) using GPU. If you are training on CPU, it might take around 30 minutes to an hour depending on the spec of your machine.

Training Deep Q Learning model (Deep Reinforcement Learning model in general) is a difficult task, and highly sensitive to changes in the hyperparameters, initializations, and changes in the environment. To avoid that, we have included a function that automatically checks whether you happen to be unlucky with your initialization, and stops the program immediately to avoid wasting time. It will print “Bad initialization. Please restart the training.”, indicating that you should re-execute the training command.

As a rule of thumb for debugging, if your rewards are not increasing consistently after a few episodes, it means that there is something wrong with your implementation. You should try to figure out what's wrong and fix it first before continuing.

After the training has completed, your agent will be tested automatically. The model of your agent will also be saved automatically at `agent/model.pt`. To manually test the saved agent with the example test cases, you can execute: `python dqn.py`. Your agent will be evaluated on 600 randomly generated test cases of the same size and specification. If you have implemented all functions correctly, you should get an average reward of ≥ 8.5 .

Grading

We follow the following grading scheme for this task:

- Avg reward ≥ 8.0 : Points = 5 (maximum point),
- Avg reward $6 \dots 8$: Points = 4
- Avg reward $4 \dots 6$: Points = 3
- Else: Point = 0.

*Point computed in the submission server.

Compute Resources

Since we are working with a problem with a larger scale, you might benefit from larger (and highly available compute resources). We recommend you to use Google Colaboratory and Tembusu cluster (if available).

Google Colaboratory is just a modified Jupyter Notebook instance running on Google Cloud Server with decent GPUs. It's free, but your instance will get reset every 12 hours. So please do make a backup of your files if you decided to use it. If you are not familiar with it, please check out the [introduction to Google Colab](#) first.

To run the agent code in Google Colaboratory (or Jupyter Notebook in general), you have to follow the following instruction:

1. At the beginning of the notebook, put a code cell to install the dependencies, e.g., torch, numpy, gym-grid-driving.
2. Open the `agent/__init__.py` and copy the content into a new cell in the Colab. Place the code before `"if __name__ == '__main__':"` in one cell, and the code after on another cell. The **argparse** will not work and should be removed. You also have to specify the **model_path** manually because **script_path** will no longer work and should also be removed.
3. To run your agent, you just have to execute the code blocks sequentially from top to bottom.

An example Colab: [HW3 Colab Example](#).

Tembusu: There is no change needed to the code apart from the fact that you have to install the dependencies yourself. Please refer to the [docker file](#) to replicate the setup manually. You will need to install python, pytorch and torchvision as per the docker file. Further, you need to setup the gym-grid-driving environment.

Submission Instructions

Please follow the instructions listed [here](#). You have to make submission as a zipped agent with the correct submission format. Please make sure to read the [frequently asked questions](#) to avoid problems.

- Please make sure that you have completed all functions that are marked with “FILL ME”.
- The maximum size of the zipped agent is 5MB.

Extra Notes:

1. Please make sure that everything that you need for the agent to run resides in the agent/ folder.
 2. Please do not print anything to the console, as it might interfere with the grading script.
 3. There will be **2 daily submission** limit for the task. Please use it wisely.
 4. The time limit of the task will be 600 seconds. If your program fails to complete on time, the system will show a “TimeoutException” error.
 5. Please make sure that you submit only what’s necessary for the agent to run.
 6. Please upload everything inside the template including MANIFEST.in
-